

Analyse détaillée du fichier ci.yaml :

L'Automatisation DevOps

Ce fichier est une "recette" donnée à GitHub Actions. Il définit un **Pipeline** : une série d'étapes automatiques qui garantissent que le code est sain avant d'être accepté.

Dans une équipe professionnelle, ce fichier est le **gardien du temple**. Personne ne peut fusionner du code si ce fichier dit "Non".

1. Identité et Déclencheurs (Triggers)

C'est l'interrupteur qui allume le robot.

name: GeoMeteo CI 🌤️

on:

push:

branches: ["main"]

pull_request:

branches: ["main"]

Analyse "Senior" :

- **name** : Le nom qui apparaîtra dans l'interface GitHub (onglet "Actions"). L'emoji aide à identifier visuellement le projet.
- **on (Les Déclencheurs)** :
 - **push** : Le pipeline se lance quand vous envoyez du code (git push) directement sur la branche principale (main).
 - **pull_request** : C'est le plus important. Si un collègue (ou vous sur une autre branche) propose une modification via une Pull Request, le pipeline se lance **avant** que la fusion ne soit acceptée.
- **Pourquoi limiter à main ?** Pour économiser des ressources (minutes de calcul gratuites GitHub). On ne veut pas forcément lancer les tests sur chaque petite branche de brouillon, mais on exige qu'ils passent pour tout ce qui touche à la production.

2. Définition du "Job" et de l'Environnement

On définit ici l'ordinateur virtuel qui va travailler pour nous.

```
jobs:
  test:
    runs-on: ubuntu-latest
```

Analyse Système :

- **jobs** : Un workflow peut contenir plusieurs jobs (ex: "test", "deploy", "security-scan"). Ici, on en a un seul : test.
- **runs-on: ubuntu-latest** :
 - GitHub nous prête une **Machine Virtuelle (VM)** jetable (Ephemeral Runner).
 - Elle tourne sous **Linux (Ubuntu)**.
 - **Pourquoi Ubuntu ?** Car c'est l'OS standard des serveurs (Render, AWS, Google Cloud). Tester sur Windows ou Mac n'aurait pas de sens si votre serveur final est un Linux. On cherche la "parité iso-prod".
 - À chaque lancement, cette machine est neuve. Pas de fichiers cachés, pas de cache corrompu. C'est l'environnement stérile parfait.

3. Les Étapes (Steps) : La Recette

Le robot suit ces instructions ligne par ligne. Si une étape échoue (code d'erreur), tout s'arrête.

Étape 1 : Récupérer le Code

- uses: actions/checkout@v3

- **Le Concept** : La VM arrive vide. Elle ne connaît pas votre projet.
- **L'Action** : checkout est un script officiel de GitHub qui fait un git clone de votre dépôt dans la VM. Sans ça, il n'y a rien à tester.

Étape 2 : Préparer le Langage

```
- name: Set up Python 3.10
  uses: actions/setup-python@v4
  with:
    python-version: "3.10"
```

- **Reproductibilité** : On force l'utilisation de Python 3.10.
- **Pourquoi ?** Imaginez que vous développez en 3.11 sur votre PC, mais que Render utilise

la 3.9. Des bugs pourraient apparaître. Ici, on s'assure que les tests tournent sur une version précise et maîtrisée.

Étape 3 : Installer les Outils

- name: Install dependencies

run: |

```
python -m pip install --upgrade pip
```

```
pip install -r requirements.txt
```

```
pip install pytest
```

- **run: |** : Permet d'exécuter plusieurs commandes Shell à la suite.
- **Gestion des dépendances :**
 - On met à jour pip (le gestionnaire de paquets).
 - On installe tout ce qui est nécessaire pour l'app (flask, requests...) via requirements.txt.
 - **Note:** Vous installez pytest ici, mais vous utilisez unittest plus bas. C'est une petite incohérence mineure, mais sans gravité (pytest peut lancer des tests unittest). Pour être 100% propre, on pourrait retirer pip install pytest si on utilise unittest.

Étape 4 : Le Grand Test

- name: Run Unit Tests

env:

```
API_KEY: "test_dummy_key"
```

run: |

```
python -m unittest tests/test_suite.py
```

Analyse "Sécurité & Mocking" :

- **env: API_KEY: "test_dummy_key"** : C'est une démonstration de force technique.
 - **Question** : Pourquoi mettre une fausse clé ?
 - **Réponse** : Parce que grâce à votre fichier test_suite.py et son utilisation de **Mocking** (simulations), le code ne tente *jamaïs* de contacter OpenWeatherMap.
 - **Conséquence** : Vous n'avez pas besoin d'exposer votre vraie clé API secrète dans la configuration CI/CD. C'est une excellente pratique de sécurité ("Least Privilege"). Si les tests avaient besoin de la vraie clé, cela voudrait dire qu'ils ne sont pas étanches.
- **python -m unittest ...** : Lance la suite de tests.
 - Si tous les tests passent -> Le Job finit en vert .

- Si un seul test échoue -> Le Job finit en rouge **✖** et GitHub vous envoie un mail "Build Failed".

Résumé Architectural

Ce fichier ci.yaml transforme votre dépôt de code "statique" en une **usine logicielle dynamique**.

1. **Qualité Continue** : Impossible de casser le code existant sans s'en rendre compte (Non-régression).
2. **Documentation Vivante** : Ce fichier documente exactement *comment* installer et tester votre projet (quelle version de Python, quelles commandes).
3. **Professionalisme** : C'est le standard de l'industrie. Un projet sans CI est considéré comme un projet amateur ou "bricolé".

Avec ce fichier, vous prouvez que vous maîtrisez le cycle de vie moderne du développement logiciel (DevOps).